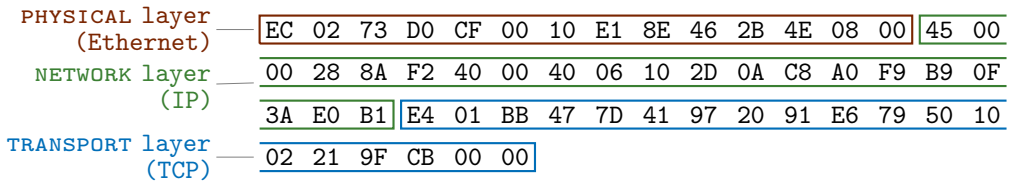
Argomenti necessari:

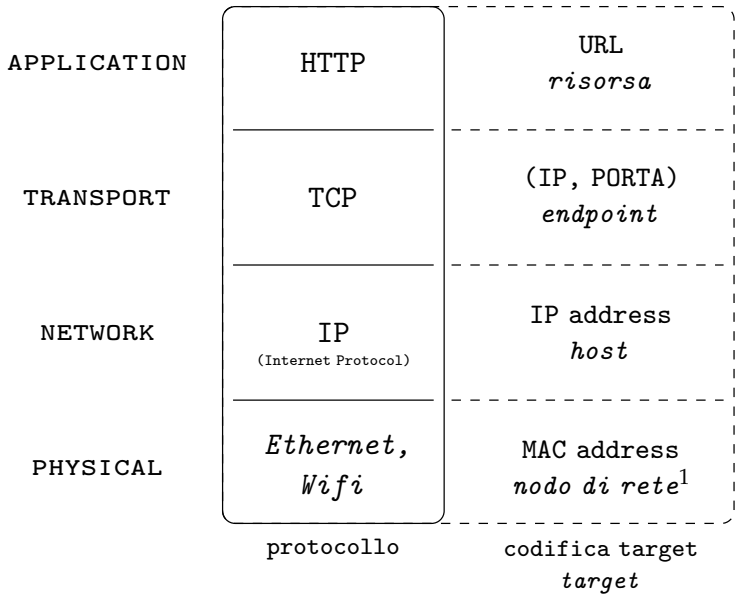
- Thread.
- Socket.

1 Lo stack TCP/IP con HTTPs

È utile avere un'idea generale di ciò che avviene durante una comunicazione in rete. In questo capitolo approfondiremo il protocollo HTTP del livello *applicazione*. Appartenendo al livello più alto dello stack a quattro livelli, per ogni comunicazione operano *"al di sotto"* del livello *applicazione* altri tre livelli. Questo significa che in una comunicazione concreta ciascun livello utilizza un protocollo specifico utilizzando meccanismi di identificazione propri. Questo si traduce nella pratica in un pacchetto di byte composto nel seguente modo:



Dove nel caso del protocollo HTTP i protocolli usati sono i seguenti, con relativi *target* e relativa *codifica*:



Come si può notare dallo stack, il target del protocollo indica il tipo di obbiettivo che ogni protocollo ha in una comunicazione. Nel protocollo TCP l'obbiettivo è un altro *endpoint*, nel protocollo IP un altro *host*, nel protocollo HTTP invece l'obbiettivo è una *risorsa*.

Guardando il pacchetto HTTP possiamo notare come non siano presenti dei byte dedicati al livello applicazione. Questo accade in quanto il livello *applicazione* è un livello *astratto*, ovvero i dati che invia sono inclusi nel livello trasporto.

[0]Come una Network Interface Card (NIC), ovvero la scheda di rete.

Cos'è una risorsa. Come abbiamo già detto, la *risorsa* è l'obbiettivo del protocollo HTTP. Ma cos'è una risorsa? Nello standard RFC3986[1] è definita come segue:

> [...] il termine ''risorsa'' è utilizzato in senso generale per indicare qualsiasi elemento identificabile tramite un URI. Esempi familiari includono un documento elettronico, un'immagine, una fonte di informazioni con uno scopo coerente (ad esempio, ''le previsioni del tempo di oggi per Los Angeles''), un servizio (ad esempio, un gateway da HTTP a SMS) e una raccolta di altre risorse. Una risorsa non è necessariamente accessibile tramite Internet; ad esempio, anche gli esseri umani, le società e i libri rilegati in una biblioteca possono essere risorse. [...]

Come si può notare da tale definizione, le risorse non sono accessibili esclusivamente tramite Internet.

Codifica delle risorsa: URI. Le risorse vengono codificate con lo standard Uniform Resource Identifier (URI). Tale codifica indetifica una risorsa come una stringa, ovvero una sequenza di caratteri codificati in US-ASCII. Nello standard RFC3986 la URI viene definita come:

> A Uniform Resource Identifier (URI) is a compact sequence of characters that identifies an abstract or physical resource.

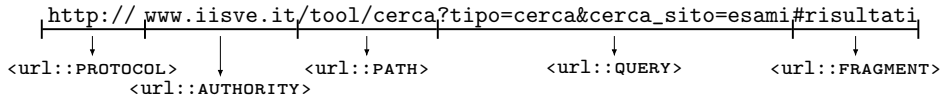
Nel caso del protocollo HTTP viene usato un sottoinsieme delle URI, gli Uniform Resource Locator (URL).

Uniform Resource Locator (URL). Il termine *locator* nell'acronimo URL sta ad indicare che nella stringa viene anche definita la modalità con cui si dovrà accedere alla risorsa, e tale modalità è il protocollo. Lo standard URI non è stato creato appositamente per il protocollo HTTP, infatti le URL possono essere usate anche con altri protocolli (ftp:// o ssh://).

URL nel protocollo HTTP. Le URL utilizzate nel protocollo HTTP (e HTTPS) specificano un modo di accedere alla risorsa che coinvolge due operatori:

- L'operatore *client* che chiede la risorsa.
- L'operatore *server* che fornisce la risorsa.

Ecco un esempio di URL completa con protocollo HTTP:



- ‹url::PROTOCOL›: protocollo della comunicazione a livello applicazione.
- ‹url::AUTHORITY›: l'autorità che possiede la risorsa.
- ‹url::QUERY›: lista di attributi chiave:valore separati da &.
- ‹url::FRAGMENT›: riferimento ad una sezione della pagina HTML.

Campo ‹url::query›. Il campo ‹url::QUERY› è suddiviso in parametri aventi la seguente forma:

```
chiave:valore
```

Possono esservi più parametri, in tal caso questi sono suddivisi dal carattere &. La sintassi è quindi la seguente:

[1] https://datatracker.ietf.org/doc/html/rfc3986.

```
?<nome_p1>=<valore_p1>[&<nome_p2>=<valore_p2>...]
```

dove le parentesi quadre indicano che possono esserci uno o più parametri.
***Attenzione*:** ‹`url::QUERY`› non è obbligatorio.

Campi obbligatori della URL. Gli unici campi dell'`URL` che non sono opzionali sono ‹`url::PROTOCOLLO`› e ‹`url::AUTHORITY`›.

Nota su authority. ‹`url::AUTHORITY`› non è da confondere con il dominio, anche se spesso nei browser vediamo solamente quest'ultimo. In modo esteso essa è definita nel seguente modo:

```
user:password@host:porta
```

Nel protocollo `HTTP`, `user` e `password` sono vietati, perciò potrà essere solamente: `host:porta`. Per quanto riguarda `porta`, anche questo è spesso omesso in quanto impostato automaticamente ad `80` per l'`HTTP` e `443` per l'`HTTPS`.
Attenzione: Un server può comunque accettare comunicazioni `HTTP` o `HTTPS` su porte differenti da 80 e 443 rispettivamente. In tali casi `porta` dovrà essere specificato, come ad esempio durante la fase di development di siti web si utilizzano server locali in ascolto sulla porta 3000 (`https://localhost:3000`).

2 Protocollo HTTP(s)

Il protocollo HyperText Transfer Protocol (`HTTP`)[2] è un protocollo applicazione utilizzato da due operatori per eseguire operazioni su una risorsa definita dalla `URL`:

- Due operatori, il *`client`* ed il *`server`*.
- Una risorsa, definita dalla `URL`.
- Un metodo, ovvero la tipologia di operazione da fare sulla risorsa.

Tali operatori comunicano a livello di trasporto con un socket `TCP`.

Panoramica di una comunicazione HTTP Il protocollo `HTTP` è un ciclo che parte dal *`client`*, il quale ha necessità di effettuare un'operazione su una risorsa posseduta da un altro operatore raggiungibile tramite rete internet, detto *`server`*. Tale operazione può essere sia di modifica o di sola acquisizione.

1. Il *`client`* genera un documento testuale chiamato *request* (`REQ`): qui dentro mette tutte le informazioni necessarie per descrivere l'operazione che vuole effettuare sulla risorsa:
   - ‹`req::url::PATH`›: specifica il *nome* della risorsa posseduta dal *`server`*;
   - ‹`req::METHOD`›: La tipologia di operazione;
   - ‹`req::url::QUERY`› e ‹`req::BODY`›: i dati[3] che servono per effettuare tale operazione;
   - ‹`req::HEADER`›: altre informazioni di configurazione come, ad esempio, la codifica utilizzata per i dati nel ‹`req::BODY`› o lo stato di autenticazione.
2. La `REQ` viene inviata dal *`client`* al *`server`*.
3. Il *`server`* riceve la `REQ`.

[2] In questa dispensa non abbiamo bisogno di distinguere tra `HTTP` e `HTTPS`, per cui useremo solamente il termine `HTTP`, valido per entrambi i protocolli.
[3] Esempio: nel ‹`req::BODY`› vanno username e password in un form di login; in ‹`req::url::QUERY`› va il testo da ricercare quando si effettua una ricerca.

4. Il *server* legge ‹req::url::PATH› per trovare la risorsa cui si riferisce il *client*:
   - Se ‹req::url::PATH› identifica una risorsa che non esiste, allora genera una *response* (RES) con errore [404, NOT FOUND], e la invia al *client*.
   - Se ‹req::url::PATH› identifica una risorsa che esiste, prosegue allo step 5.
5. Il *server* legge ‹req::METHOD› e **verifica** che la funzione che gestisce tale ‹req::METHOD› su tale risorsa sia implementata:
   - Se non è implementata il *server* genera una RES con ‹res::CODE› uguale a [405, METHOD NOT ALLOWED] e la invia al *client*.
   - Se invece è implementata prosegue allo step 6.
6. Il *server* ora **verifica** che il *client* abbia i sufficienti permessi per effettuare l'operazione di tipo ‹req::METHOD› su tale risorsa.
   - Se il *client* non è autorizzato, il *server* genera una RES con errore [401, UNAUTHORIZED] e la invia al *client*.
   - Se il *client* è autorizzato prosegue allo step 7.
7. Il *server* legge i dati contenuti in ‹req::QUERY› e/o ‹req::BODY›:
   - Se i dati non sono come se li aspetta (*validi*), il *server* genera una RES con ‹res::CODE› uguale a [400, BAD REQUEST] e la invia al *client*.
   - Se i dati sono come se li aspetta prosegue allo step 8.
8. Il *server* effettua infine l'operazione sulla risorsa:
   - In questa fase se si generano degli errori non è più a causa della REQ ma dei possibili bug presenti nell'implementazione della funzione che gestisce la REQ nel *server*. In caso di errori, il *server* genera una RES con ‹res::CODE› uguale a [500, SERVER ERROR] e la invia al *client*.
   - Se tutto va liscio il *server* genera una RES con ‹res::CODE› uguale a [200, OK] e la invia al *client*.

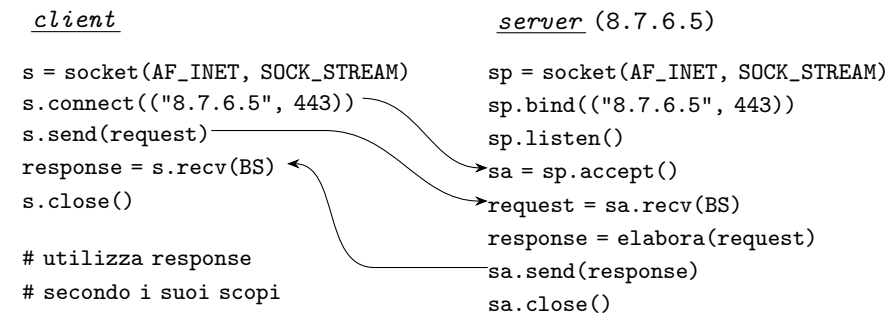
Quando il *client* riceve la RES:

- Verifica l'esito attraverso ‹res::CODE›.
- Se aveva richiesto di ricevere dei dati dal *server* legge ‹res::BODY›[4] decodificandolo secondo quanto indicato negli ‹res::HEADER›.

Infine il *client* utilizza i dati ricevuti secondo i propri scopi. Esempi semplici: se era un'operazione di *acquisizione* dati può mostrare i dati ricevuti (come una pagina HTML), se era un'operazione di modifica della risorsa può avvertire che l'operazione nel *server* è andata a buon fine, o in caso contrario comunicare l'eventuale errore.

2.1 Protocollo HTTP nei vari livelli dello stack

Livello di trasporto: socket TCP. Il protocollo HTTP agisce a livello di trasporto adottando un socket TCP il quale funge da ponte comunicativo tra il *client* ed il *server*. Il *client* si connette, il *server* è in ascolto per eventuali connessioni in ingresso:



[4] Non esiste ‹QUERY› nella response.

Dove BS è una variabile intera a cui diamo un valore generale in quanto non è centrale durante questa spiegazione.

La risorsa è sempre posseduta dal *server*. Questo è molto importante. Si immagini il *server* composto almeno da due elementi principali:

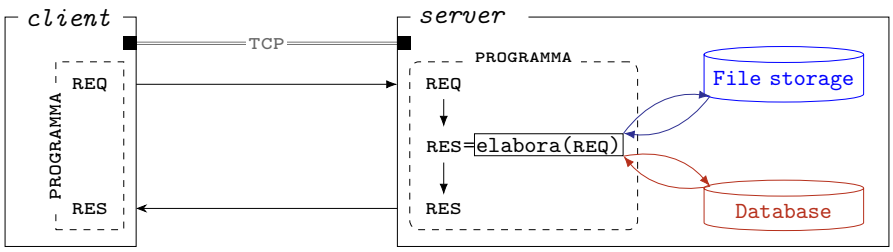
- Un programma che esegue l'endpoint HTTP, ovvero un socket TCP in ascolto di uno o più *client*.
- Un sistema di archiviazione (persistenza):
  - Un programma che esegue il Database.
  - Un programma che esegue il File storage.

Al contempo *client* è un programma che esegue un socket TCP.

Livello applicazione: ciclo *request* e *response*. Dato che è il *server* a fornire la risorsa al *client* che la richiede, il protocollo HTTP a livello applicazione funzionerà come segue:

1. Il client genera la *request* (REQ): un documento testuale codificato in US-ASCII.
2. Attraverso il socket TCP invia REQ al *server*.
3. Il *server* riceve REQ e la elabora generando una *response* (RES), sempre codificata in US-ASCII.
4. Il *client* riceve RES e la utilizza per i propri scopi.

Questo è sintetizzato nella seguente immagine:



Si può notare come sia centrale la funzione elabora(REQ) la quale restituisce la RES a seconda della REQ creata dal client. In questa funzione, è possibile ma non obbligatorio che il *server* interagisca con il database ed il file storage.

2.2 Scambio di dati tra client e server.

Il *client* ha due possibilità per inviare dati dinamici al server:

- Il campo ‹req::URL::QUERY›: parametri codificati nell'URL. In ‹req::URL::QUERY› vengono mandati dati quando questi sono pochi e hanno lunghezza (in byte) piccola.

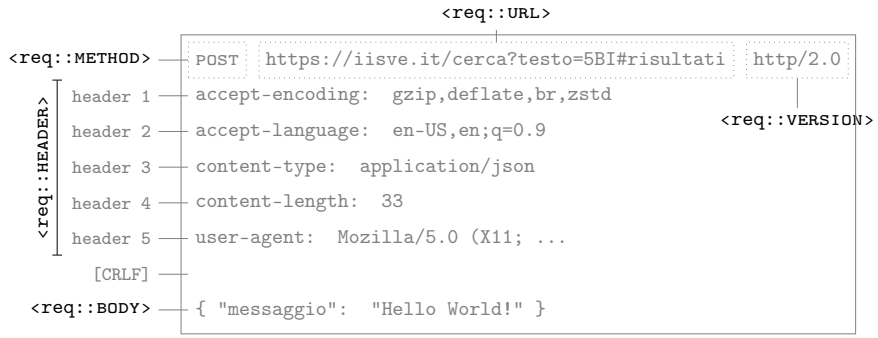
- Il campo ‹req::BODY›: qui va qualsiasi dato, da una pagina html codificata in UTF-8 ai byte di un'immagine jpg.

Il *server* invece invia i dati solamente attraverso il ‹req::BODY›.

La request. La request (REQ) è un file testuale codificato in US-ASCII strutturato nel seguente modo:

- La prima riga contiente i seguenti campi separati da un carattere spazio:
  - ‹req::METHOD›: la tipologia di operazione sulla risorsa[5].
  - ‹req::URL›: la URL. I campi della URL nella REQ sono idenficati così: ad esempio ‹url::QUERY› diventa ‹req::url::QUERY›.
  - ‹req::VERSION›: la versione del protocollo HTTP (es: «http/2.0»).
- ‹req::HEADER›: dalla seconda riga fino al carattere [CRLF][6]: ciascuna riga rappresenta un *header*, campi di configurazione in forma: «nome header:valore header».
- ‹req::BODY›: campo che può essere opzionale (non esiste se ‹req::METHOD› è GET). Contiene dati inviati dal *client* al *server* codificati (il tipo di codifica è descritto nell'header content-type, si veda Tabella 1).

La seguente immagine mostra un esempio con body codificato in JSON:



La response. La response (RES) è un file testuale codificato in US-ASCII strutturato nel seguente modo:

- La prima riga contiente i seguenti campi separati da un carattere spazio:
  - ‹res::VERSION›: la versione del protocollo HTTP («http/2.0»).
  - ‹res::CODE›, un codice numerico che identifica l'esito della REQ.
  - ‹res::REASON›, ovvero la spiegazione testuale dell'esito.
- ‹res::HEADER›: come nella REQ, dalla seconda riga fino al carattere [CRLF].
- ‹res::BODY›: campo può essere opzionale, contiene i dati inviati dal *server* al *client*.

[5]Possibili valori sono: (GET/POST/DELETE/OPTIONS/PUT/PATCH)

[6]Indica una riga composta solamente dai caratteri Carriage Return (CR) e Line Feed (LF), in pratica è una riga vuota.

Tabella 1: Vari possibili valori per l'header `content-type`.

Contenuto body	`content-type`
Immagini jpg	`image/jpg`
Immagini png	`image/png`
Pagina HTML	`text/html`
File CSS	`text/css`
File Javascript	`text/javascript`
File binari	`application/octet-stream`

Request e response: body. ‹BODY› è il *corpo* della REQ o RES. Esso trasporta i dati che il *client* vuole trasmettere al *server* e viceversa. La codifica del ‹BODY› è definita negli ‹HEADER›:

- `content-length`: numero di byte del body (sia nella REQ che nella RES).
- `content-type`: codifica del body, si veda Tabella 1.

ATTENZIONE: Il body non è presente nelle REQ quando ‹req::METHOD› è GET.

Response: status code. Il campo ‹res::CODE›, detto *status code* è un codice numerico di tre cifre, presente nella prima riga della RES. Esso indica l'esito della richiesta e separato da uno spazio c'è ‹res::REASON›, ovvero la descrizione testuale dello *status code*. Di seguito vengono elencati i vari range di *status code*:

- Esito *successful*, range `200-299`: indica che la richiesta è stata elaborata con successo dal *server*.
- Esito *client error*, range `400-499`: indica che la richiesta del *client* non può essere soddisfatta perché non valida. Un esempio comune è quando il *client* chiede una pagina web che non esiste: `404 - Not Found Error`.
- Esito *server error*, range `500-599`: indica che la richiesta del *client* non può essere soddisfatta a causa di errori verificatisi nel *server*.

3 Metodi

La definizione di metodo, contenuti nel campo ‹req::METHOD›, è la seguente:

> Un metodo definisce il tipo di operazione che il *client* richiede al *server* di effettuare sulla risorsa definita dall'URL.

Metodo GET. Il metodo GET definisce una richiesta per ottenere una risorsa definita dall'URL. Peculiarità di una request GET:

- ‹req::BODY› è vuoto.
- L'unico modo per il *client* di inviare dati è il campo ‹req::url::QUERY›.
- Il *server* invia i dati in ‹res::BODY›. La codifica del body è definita in un header di ‹res::HEADER› dal nome `content-type`.

Esempi di utilizzo:

- Navigazione web: ogni pagina viene ottenuta tramite request GET.
- Immagini: ogni immagine visualizzata in una pagina viene ottenuta dopo richiesta GET.
- Ricerche: il testo da ricercare viene passato dal *client* attraverso i parametri di ‹req::url::QUERY›.

Metodo POST. Il metodo POST definisce una richiesta per modificare, creare, eliminare la risorsa definita in ‹req::URL›.

- ‹req::BODY› non è vuoto e viene usato per inviare i dati. Può contenere dati strutturati (XML, JSON), immagini, testo, eccetera.
- ‹req::QUERY› solitamente è inutilizzato.

Esempi di utilizzo:

- Autenticazione: quando un utente si autentica per effettuare il *login* in un sito la richesta è POST.
- Modifica stato dello studente: quando si modifica se uno studente è presente, presente fuori aula, ecc...
- Quando si carica un'immagine di profilo.
- Quando si effettua il logout.
- Quando si crea un contentuto su un social network o sito in generale.